Summary of System Interfaces for Agents

Rashon Poole (rashonp), Boe Zhou (boezhou), Nikolai Zhitkov (zhitkov), Roger Zhou (zimingzh)

Section 1: Research Direction of the Field

Traditional computer systems are built for humans and conventional software, so AI agents must navigate interfaces and abstractions that aren't designed for them. This leads to inefficiencies, unreliable interactions, and limited feedback. CAKE and DMI tackle a broader research challenge. They focus on designing computer systems that treat AI agents as first-class users instead of requiring them to adapt to existing systems. Earlier systems research established abstractions such as compiler intermediate representations, optimization passes, and operating-system interfaces [1,2,3]. Meanwhile, AI-agent research focused on improving agents within existing environments. GUI agents such as Agent S and UI-TARS improved interface navigation [5,7], while KernelBench and KernelEvolve advanced GPU kernel optimization [6,7]. However, these approaches remained constrained by interfaces designed for traditional users.

Both papers introduce agent-native abstractions that separate AI reasoning from low-level execution. CAKE redesigns GPU compiler infrastructure through structured representations, interpretable diagnostics, and tools for compiler improvement. DMI replaces fragile GUI interaction sequences with declarative operations, letting agents express their intentions while the system handles execution. Their results show promising gains in efficiency and reliability, making them early examples of system–agent co-design. Both works are relatively recent (July 2026 and April 2026), so follow-up work has yet to be formally established.

Section 2: Lecture Papers

CAKE. CAKE addresses the difficulty AI agents face when optimizing GPU kernels using programming interfaces that either hide important hardware details or expose excessive low-level complexity. It introduces Cake IR, a structured representation that exposes hardware scheduling decisions while automating error-prone implementation details. CAKE also provides compiler-based verification, performance diagnostics, and a mechanism for evolving the compiler harness through recurring failures and newly identified capabilities. Evaluated primarily on NVIDIA B200 GPUs, CAKE achieves a median best performance of 1.144× the tuned FlashML baseline on Flash-KMeans,

compared with 0.928× for direct CUDA/PTX under matched conditions. Additional experiments demonstrate improvements across attention and other GPU workloads. Limitations include untested portability beyond NVIDIA GPUs, predominantly B200-based performance evidence, incomplete static analyses and performance models, and continued human oversight of compiler evolution.

DMI. DMI addresses the difficulty AI agents face when interacting with human-oriented graphical interfaces, which require lengthy navigation sequences and repeated visual observations. It introduces a declarative interface with three primitives: Access, State, and Observation. By separating high-level semantic planning from deterministic GUI navigation and interaction, DMI reduces the need for error-prone low-level operations. Evaluated on 27 single-application tasks across Microsoft Word, Excel, and PowerPoint using UFO-2, DMI increases task success from 44.4% to 74.1%, reduces average LLM calls by 43.5%, and decreases completion time on successful runs from 392 to 239 seconds. An ablation study confirms that the declarative interface, rather than navigation information alone, is the primary driver of improvement. Limitations include incomplete support for complex GUI interactions, dependence on accessibility APIs, application-specific modeling effort, and challenges with dynamic interfaces.

Section 3: Lecture Discussion

- **How much does CAKE's compiler evolution contribute, and can agents game it?** Its contribution is untested, since the paper has no ablation and only compares Cake IR against direct CUDA.
- **Can kernel optimizations be formally verified (Lean, eBPF-style)?** A verification agent that checks for known hacks can be created, plus human merge gates and more tests. The counterpoint was that tests always lag behind smarter AIs, so a formally verified IR is needed.
- **How vendor-agnostic is CAKE?** Only partly, but lowering to non-NVIDIA hardware would take substantial engineering.
- **How does DMI's forest differ from the original graph?** Shared subtrees are stored once and reached through reference IDs, so each control keeps a unique path without duplicated subtrees, which keeps the prompt small.
- **Are DMI's tasks too simple, and can its token cost be amortized or cached?** The tasks are short, the "single LLM call" is one core call, and failed runs are excluded from the cost metrics. The navigation graph already acts as a cache, but dynamic UI outside it forces a step-by-step fallback to the GUI, which limits amortization on long tasks.
- **Should we retrofit human interfaces or redesign them for agents?** Thinking of redesigning OS and web interfaces around one representation that serves both humans and agents, instead of reverse-engineering GUIs paper by paper.

Need to keep in mind that DMI-style retrofits may not keep pace with stronger models.

Section 4: Future Research Direction

CAKE and DMI suggest a broader shift toward designing computer systems that treat AI agents as first-class users. Both demonstrate that agent performance can improve through better system abstractions without changing the underlying language model. However, important questions remain about how agent-facing interfaces should be constructed, how responsibilities should be divided between agents and systems, and whether such interfaces can evolve safely. A promising research direction is Self-Extending Declarative Interfaces (SE-DMI), which challenges the assumption that agent-facing operations must be predefined. Building on DMI's declarative interface and CAKE's approach to system co-evolution, SE-DMI would enable agents to discover application functionality, synthesize reusable semantic operations, and automatically adapt these operations when applications change. Each synthesized operation would include explicit preconditions, postconditions, and validation mechanisms to support reliable execution.

The main research challenges involve verifying semantic correctness, detecting interface changes, controlling unintended side effects, and balancing discovery costs against long-term reuse. SE-DMI could be evaluated against existing GUI-agent and DMI baselines using task success, manual setup effort, verification accuracy, capability reuse, and adaptation to application updates. This direction would extend agent-native interface research from predefined abstractions toward dynamically constructed, verified, and evolving interfaces, while retaining conventional system mechanisms for reliable execution.

Section 5: AI Usage

AI provided the most assistance in sections 1 and 2, but limited assistance in 4. LLMs have abundant knowledge and can web-search, so they can complete tasks like surveying a field very well. Here I asked questions like “What broader change in computer systems research do CAKE and DMI represent?” and “how earlier research led to this direction?” For section 2, when provided with the PDFs of the papers, it can also summarize and find the important parts well. Here I used it to check my own understanding of the papers and asked questions like “Of CAKE's major design choices, which is most responsible for its improvements?”. However, one issue was that when prompted, LLMs will find issues with something even if it does not exist. So because of that, limitations of the papers were found manually. LLMs ability for section 4 was a bit

limited compared to the others. From my experience, the LLM was not able to propose good future research topics and solutions.

Citations

- [1] Brinch Hansen (1970) – The Nucleus of a Multiprogramming System
- [2] Chen et al. (2018) – TVM: Automated Compiler for Deep Learning
- [3] Lattner et al. (2021) – MLIR: Compiler Infrastructure
- [4] Agashe et al. (2025) – Agent S: AI Computer-Use Agent
- [5] Qin et al. (2025) – UI-TARS: GUI Interaction with AI Agents
- [6] Ouyang et al. (2025) – KernelBench: AI-Generated GPU Kernels
- [7] Liao et al. (2026) – KernelEvolve: Automated GPU Kernel Optimization